

Advanced JavaScript Event Loop Master Guide

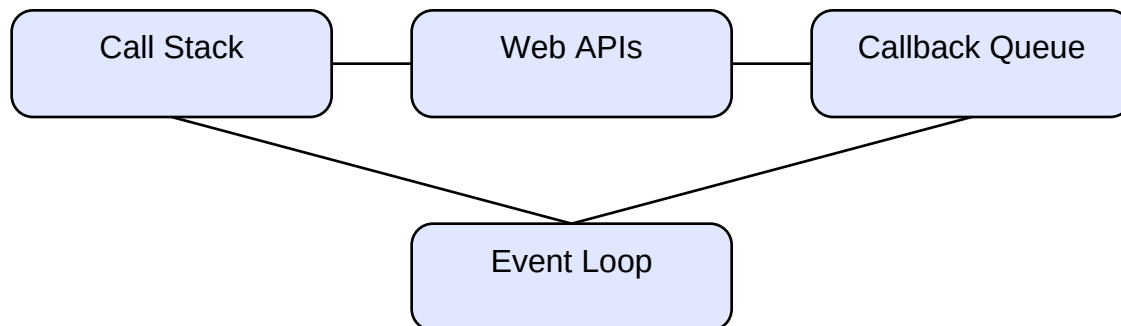
This PDF transforms the original notes into a full learning resource with detailed explanations, visualization diagrams, execution flow analysis, mental models, and practical examples.

Topics Covered: JavaScript Runtime Call Stack Web APIs Callback Queue Microtask Queue Event Loop Promises async / await Blocking vs Non-blocking code Execution Priority

1. JavaScript Runtime Architecture

JavaScript itself is single-threaded. That means it can execute only one instruction at a time. However, browsers and Node.js provide additional systems that make asynchronous programming possible.

The runtime architecture mainly consists of: **Call Stack** → Executes synchronous code **Web APIs** → Handle async tasks outside JS engine **Queues** → Store completed callbacks **Event Loop** → Coordinates execution



2. Call Stack

The Call Stack works like a stack of plates.

Rules: Function calls are pushed into the stack. When execution finishes, they are popped out. The topmost function always executes first.

```
function first() {
```

```

    console.log("First");
  }

  function second() {
    first();
    console.log("Second");
  }

  second();

```

Execution Flow: second() → first() → console.log → return → console.log

3. Web APIs

Functions like `setTimeout()`, `fetch()`, and DOM events are not handled directly by the JavaScript engine. Browsers provide Web APIs to process them asynchronously.

When async work completes, callbacks move into queues.

```

console.log("Start");

setTimeout(() => {
  console.log("Timeout Done");
}, 2000);

console.log("End");

```

Output:

Start

End

Timeout Done

4. Event Loop

The Event Loop continuously checks: Is the Call Stack empty? Are there tasks waiting in queues? If yes, it moves tasks back into the Call Stack.

Priority	Queue	Examples
1	Microtask Queue	Promise.then(), await continuation
2	Callback Queue	setTimeout(), setInterval()

5. Microtask Queue vs Callback Queue

Promise callbacks have higher priority than normal callbacks.

Microtasks always execute before Callback Queue tasks.

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Actual Output:

Start
End
Promise
Timeout

6. async / await Deep Explanation

The await keyword does NOT block the entire JavaScript engine. It pauses only the current async function.

Internally: await converts continuation into a microtask. The remaining code executes later through the Event Loop.

```
async function demo() {
  console.log("A");

  await Promise.resolve();

  console.log("B");
}

demo();

console.log("C");
```

Output:

A
C
B

7. Blocking vs Non-blocking Code

Heavy synchronous loops block the Call Stack. While blocked: UI freezes Callbacks cannot execute Event Loop cannot continue

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

for(let i = 0; i < 10000000000; i++) {}

console.log("End");
```

Even with 0ms delay, the timeout waits until blocking code finishes.

8. Mental Model

Call Stack → Executes active code

Web APIs → Perform async work externally

Queues → Store completed tasks

Event Loop → Re-inserts tasks into execution

9. Final Summary

JavaScript achieves asynchronous behavior through collaboration between: Call Stack Web APIs

Microtask Queue Callback Queue Event Loop **Most Important Rule:**

Microtasks execute before Callback Queue tasks.

Prepared as an advanced learning guide from the uploaded PDF study notes.